Report, COMP.SE.140 exercise 1 - Eemil Pirinen H292061

1 Platform

The application was implemented with the following hardware and operating system

- 16GB RAM
- AMD Ryzen 5 3600
- AMD Radeon RX 5700 XT
- Windows 11 Pro 24H2

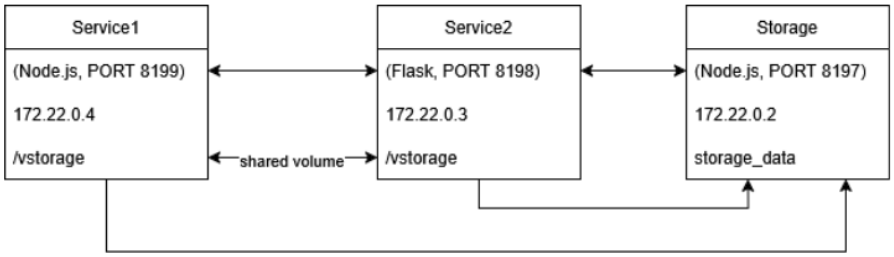
Additionally the application was tested on a MACH-WX9

- Intel i7-8550U
- 16GB RAM
- OS: Arch Linux x86_64
- Kernel: Linux 6.16.8-arch3-1

Docker and Docker Compose versions used were the following

- Docker version 28.1.1, build 4eba377
- Docker Compose version v2.35.1-desktop.1

2 Diagram of services



3 Analysis of status record contents

The status records produced by the services contained the following information

- Current UTC time when the request was served (format: 2025-09-03T12:06:18Z)
- Uptime, which represents the time since the container was started (format: X.XX hours)
- Free disk space in root (format: X MB)

In Service1 the uptime was measured using process.uptime() whereas in Service2 the same was achieved by calculating (time.time() - start_time). These uptimes could perhaps be useful when monitoring the live status of the services (as it does in the application), but not really for anything else. This is because the uptimes record the time the service process has been running but loses its usefulness if the containers are voluntarily or involuntarily shut down. However, I could see some possible uses for the tracked uptime considering it was reported together with a datetime. Individual uptimes don't mean much but being able to follow where and when the uptime has restarted and combining it with the datetime gives somewhat useful information regarding the services.

Free disk space in root was calculated in Service1 with CheckDiskSpace(“/”) and in Service2 with shutil.disk_usage(“/”).free. As far as I understood it the disk space at (“/”) refers to the container's main disk. During development I came to realize that the amount of data that was written during the execution of these 3 services was so small that changes to the free disk space were minimal at best. Therefore I believe that tracking such contents isn’t very useful unless in the case of this exercise but could be useful on larger services to monitor disk availability and to avoid crashes due to a full disk.

4 Analysis of persistent storage solutions

./vstorage:/vstorage
I found the vstorage implementation useful when wanting to inspect the log data while the containers were not running and overall accessing the storage was easy as it existed as a physical file within the root of the application. However clearing the vstorage logs were a bit complicated as they were not automatically cleared when running “docker compose down -v”. Vstorage was also very easy to access from multiple different services. Overall I found the vstorage implementation being somewhat useful together with the other persistent storage method but vstorages status as being partly tied to the host filesystem and partly managed by Docker was kind of confusing and the fact that it’s less portable makes it much less useful.

storage_data:/data
The biggest downside for the other persistent storage method implemented with the storage service was that it was harder to inspect than vstorage as it only lived inside docker and was not accessible if the containers were not running. However, this also made it an easier concept to grasp than the vstorage implementation, as it was entirely managed by Docker. Also the fact that it is entirely managed by Docker makes the storage service solution much more portable compared to vstorage.

5 Instructions for cleaning up

In a case where the containers are running, the logs from both storages can be easily cleared by running the following command:

curl http://localhost:8199/clear

The instructions for this exercise also requested clearing methods for when the containers are not running. This can be achieved by running the following commands:

: > ./vstorage ***(This one is for Linux/macOS)***
Set-Content .\vstorage -Value “” **(This one is for Windows (PowerShell))**

The above mentioned commands are used to clear the vstorage. To clear the one that is maintained by the storage service, the following command is needed:

docker compose down -v

6 Retrospect

The actual implementation of the services turned out to be less demanding than initially expected. However the main difficulty for me in this exercise was in grasping what was actually demanded of me. I'm not sure if it's due to my own inexperience with Docker or for some other reason but I had a very hard time trying to understand the instructions and more importantly trying to get a clear image on what the exercise was about. I realized this as multiple times throughout the development I had difficulties in understanding if my application was doing the right thing or the right way, especially when it came to the storages.